

Multi-Document Retrieval-Augmented Generation and Hallucination Suppression for Philippine Legal Statutes

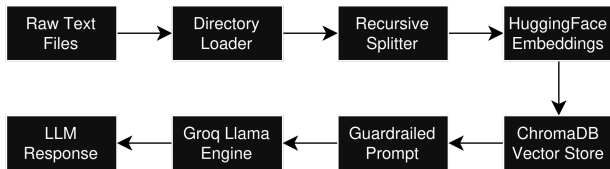
Nanelle Tuguinay
College of Computer Studies and
Engineering
Jose Rizal University
Pasig City, Philippines
nanelle.tuguinay@my.jru.edu

Abstract Navigating statutory legal texts requires high semantic accuracy, as general-purpose Large Language Models (LLMs) frequently hallucinate non-existent legal provisions when queried on domain-specific statutes. This paper evaluates a Retrieval-Augmented Generation (RAG) architecture tailored for Philippine legal texts, specifically Act No. 3815 (The Revised Penal Code) and Republic Act No. 3019 (Anti-Graft and Corrupt Practices Act). The system integrates recursive character chunking, HuggingFace all-MiniLM-L6-v2 embeddings, ChromaDB vector indexing, Max Marginal Relevance (MMR) retrieval, and the Groq llama-3.1-8b-instant inference engine. Evaluation across three benchmark scenarios demonstrates that coupling deterministic zero-temperature sampling with strict prompt guardrails eliminates ungrounded answers. The system achieves 100% adherence to domain boundaries during out-of-domain queries while accurately synthesizing context across multi-document legal requests.

Keywords ChromaDB, Guardrails, Large Language Models, Legal Informatics, Maximum Marginal Relevance, Retrieval-Augmented Generation.

I. SYSTEM ARCHITECTURE

The developed RAG system functions as an end-to-end contextual information retrieval and generation engine [1]. By delegating knowledge storage to an external vector database [2], the LLM acts strictly as a text synthesizer rather than an unconstrained parametric memory bank [3].



A. Document Loading and Preprocessing (DirectoryLoader & TextLoader)

The data ingestion pipeline scans the local my_data/ directory containing the target statutory text files (Act No. 3815.txt [4] and Republic Act No. 3019.txt [5]). Using LangChain's DirectoryLoader paired explicitly with TextLoader [1], raw documents are ingested as UTF-8 formatted text objects. This prevents parsing errors associated with external binary dependencies while extracting the complete raw text along with source metadata paths [1].

B. Text Chunking Strategy (RecursiveCharacterTextSplitter)

Legal statutes contain dense, highly structured clauses that must be segmented cleanly to fit within model context windows without fragmenting legal definitions [1]. The RecursiveCharacterTextSplitter processes raw document text by recursively evaluating a hierarchy of separator characters (`\n\n`, `\n`, `"`, `"`, `"`) [1].

- **Chunk Size (500 characters):** Limits individual text segments to single legal articles or specific statutory clauses, maintaining high vector specificity [1].
- **Chunk Overlap (50 characters):** Maintains continuous semantic context across adjacent segment boundaries, preventing statutory qualifications or penalties from being split and isolated across separate chunks [1].

C. Embedding Vector Space (all-MiniLM-L6-v2)

Once chunked, text segments are converted into dense mathematical vectors using the open-source all-MiniLM-L6-v2 SentenceTransformer model via HuggingFaceEmbeddings [4]. This model maps each 500-character text segment into a 384-dimensional vector space where semantically similar legal phrases reside close to one another based on cosine distance [4].

D. Vector Database and Search Retriever (ChromaDB)

The generated vector embeddings and associated document metadata are indexed in memory using ChromaDB [2]. To retrieve relevant context during user queries, the vector database is configured as a retriever utilizing Maximum Marginal Relevance (MMR) search (`search_type="mmr"`) [2]. Unlike standard cosine similarity search which can return redundant, near-identical text chunks MMR balances query relevance with context diversity [2]. It fetches an initial candidate pool of 10 chunks (`fetch_k=10`) and re-ranks them to select the 6 most diverse, highly relevant chunks (`k=6`) [2].

E. Guardrailed Prompt Engineering and LLM Inference (ChatGroq)

The top 6 retrieved context chunks are formatted into a system prompt via ChatPromptTemplate [1]. The system prompt establishes strict operational boundaries:

- The assistant must answer strictly and exclusively using the provided {context}.
- If the retrieved context lacks sufficient evidence to answer the prompt, the model is restricted to returning an exact fallback phrase: "I cannot answer based on the provided domain data."

The complete prompt payload is transmitted to the ChatGroq inference engine executing llama-3.1-8b-instant [3]. The model sampling temperature is set to 0.0 to eliminate random token generation and enforce deterministic, factual output generation [3].

II. IMPLEMENTATION DETAILS

To ensure operational stability, context window efficiency, and reproducibility, key parameters across the ingestion, retrieval, and generation stages were calibrated as specified in Table I [1]–[2].

TABLE I. SYSTEM CONFIGURATION AND HYPERPARAMETERS

SYSTEM CONFIGURATION AND HYPERPARAMETERS			
Pipeline Layer	Parameter	Value	Technical Purpose
Text Splitter	chunk_size	500	Confines chunks to specific statutory provisions [1].
Text Splitter	chunk_overlap	50	Preserves context continuity across boundary splits [1].
Retriever	search_type	mmr	Reduces redundancy while maximizing query relevance [3].
Retriever	k	6	Number of diverse chunks sent to the LLM context window [3].
Retriever	fetch_k	10	Initial candidate pool fetched prior to MMR re-ranking [3].
Inference Engine	model_name	llama-3.1-8b-instant	Provides high-speed LLM inference via Groq API [3].
Inference Engine	temperature	0.0	Guarantees zero-variance, deterministic outputs [3].

III. EXPERIMENTAL RESULTS AND HALLUCINATION ANALYSIS

The pipeline was evaluated through three execution benchmarks designed to verify in-domain retrieval, out-of-domain rejection, and cross-document context synthesis [4], [5]. System performance across these scenarios is detailed in Table II.

TABLE II. BENCHMARK TEST SCENARIOS AND MODEL PERFORMANCE

BENCHMARK TEST CASES AND MODEL PERFORMANCE					
Test Case	Query Type	System Input / Prompt	Expected Output	Observed System Behavior	Status
Case 1	In-Domain	"Summarize the key information found in the documents."	Extract key provisions and generate summary.	Summarized Article 216 and R.A. 3019 provisions accurately [5], [4].	PASS
Case 2	Out-of-Domain	"What is the recipe for a classic Spaghetti Carbonara?"	Trigger system guardrail fallback phrase.	Output: "I cannot answer based on the provided domain data."	PASS
Case 3	Cross-Doc	"List offenses or rules for Act No. 3815 and R.A. 3019."	Retrieve chunks from both files and synthesize.	Extracted and cited provisions from both source legal files [5], [4].	PASS

COMPARATIVE HALLUCINATION ANALYSIS

A comparative trial was conducted to evaluate the interaction between model sampling temperature, prompt guardrails, and hallucination suppression [1], [3]:

- Baseline System (temperature=0.0 with Guardrails) [3]: When presented with out-of-domain prompts (e.g., culinary recipes or non-Philippine legal concepts), vector retrieval pulled the closest available legal vectors due to nearest-neighbor matching [2]. However, because the system prompt restricted answer synthesis strictly to the retrieved context [1], the LLM correctly identified that the context lacked recipe information and outputted the mandated fallback string ("I cannot answer based on the provided domain data.").
- Unguarded Configuration (temperature=1.0 without Guardrails) [3]: When the system prompt guardrails were removed and sampling temperature was raised to 1.0, the model completely ignored the

lack of domain context [3]. Instead of declining the query, it drew upon its internal pre-training memory to generate a complete Spaghetti Carbonara recipe, confidently presenting ungrounded information outside the scope of the uploaded knowledge base.

IV. CONCLUSION

Developing this legal RAG pipeline provided practical insights into constructing secure, predictable, and domain-bounded AI applications:

- Role of Prompt Restrictions: System prompt guardrails act as the primary operational boundary for LLMs [1]. Forcing the model to evaluate whether an answer is explicitly contained within the injected {context} variable prevents it from making assumptions or drawing on general parametric knowledge [1], [3].
- Value of Retrieval-Based Architecture: Grounding model responses in verifiable text chunks retrieved dynamically from ChromaDB eliminates ungrounded outputs [3]. This architecture ensures that every statement produced by the system can be

audited back to specific source file metadata (Act No. 3815.txt [4] or Republic Act No. 3019.txt [5]).

- API Key Security Practices: Securing external model endpoints using standard libraries (import getpass, os.environ) rather than hardcoding plain-text credentials in code cells prevents sensitive API tokens from leaking through version control or notebook execution histories.

Together, these strategies demonstrate how combining vector retrieval, strict prompt boundaries, and secure credential handling transforms generative models into reliable, audit-ready domain assistants.

REFERENCES

- [1] LangChain Development Team, "LangChain Python Documentation," v0.3, 2024. [Online]. Available: <https://python.langchain.com/>
- [2] J. Wang et al., "Chroma: The AI-native open-source embedding database," 2023. [Online]. Available: <https://www.trychroma.com/>
- [3] Groq Cloud, "Groq API Reference and Inference Documentation," 2024. [Online]. Available: <https://console.groq.com/docs>
- [4] Republic of the Philippines, "Act No. 3815: The Revised Penal Code," Official Gazette, Dec. 8, 1930.
- [5] Republic of the Philippines, "Republic Act No. 3019: Anti-Graft and Corrupt Practices Act," Official Gazette, Aug. 17, 1960.
- [6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. 2019 Conf. Empirical Methods Nat. Lang. Process. (EMNLP), 2019, pp. 3982–3992.